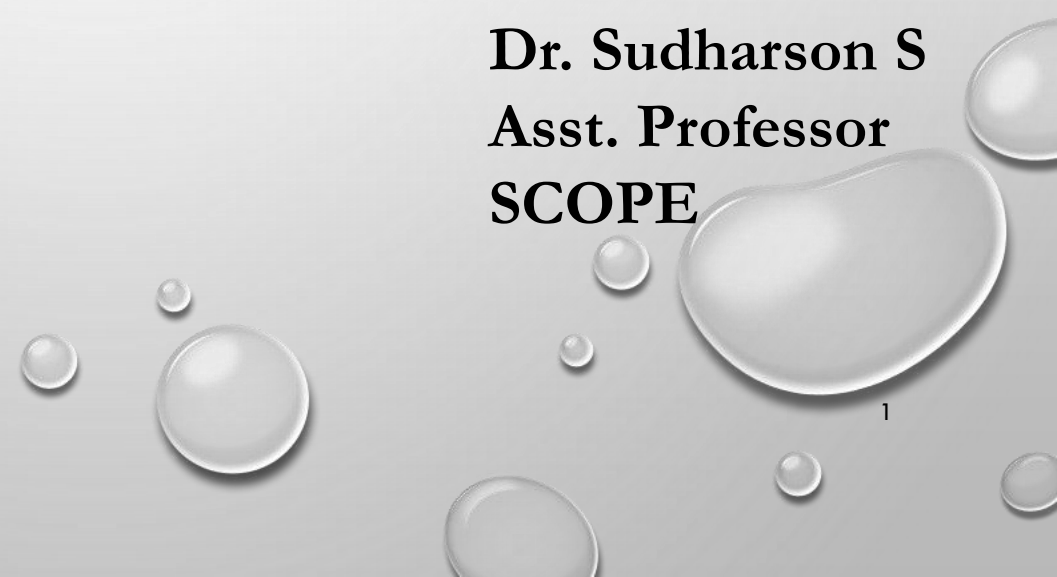




OPERATING SYSTEM

BCSE303L

Dr. Sudharson S
Asst. Professor
SCOPE



Module:5	Memory Management	7 hours
Main memory management, Memory allocation strategies, Virtual memory: Hardware support for virtual memory (caching, TLB) – Paging - Segmentation - Demand Paging - Page Faults - Page Replacement -Thrashing - Working Set.		

TLB

In operating systems, the **Translation Lookaside Buffer (TLB)** is a **small, very fast memory** that helps the CPU find the physical memory address quickly when using virtual memory.

Why TLB is needed

In systems that use **paging**, the CPU generates a **logical (virtual) address**.

To access the real memory location, the system must **translate the logical address into a physical address** using the **page table**.

The **page table is stored in main memory**.

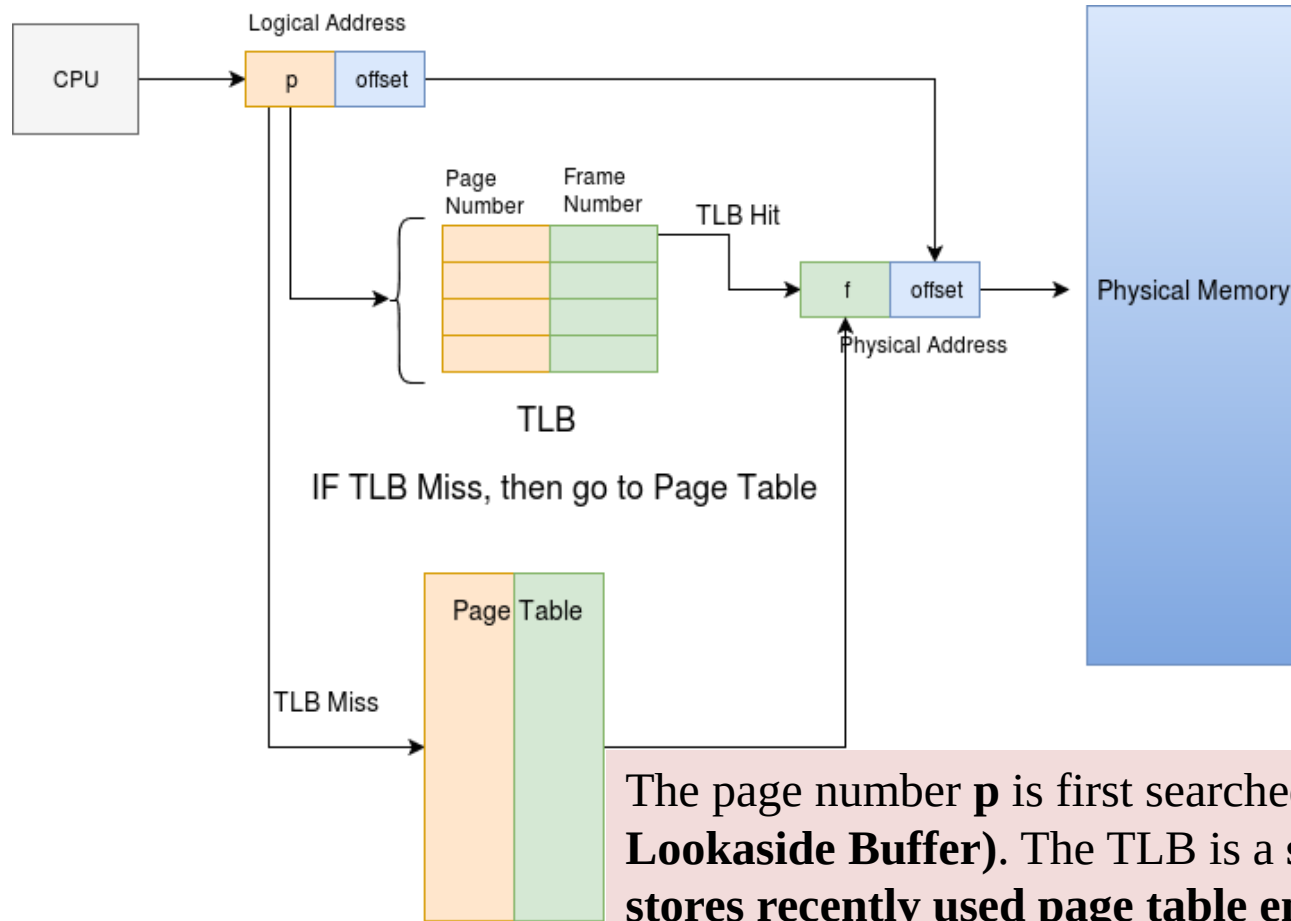
So normally the system would need:

1. One memory access to read the **page table entry**
2. Another memory access to read the **actual data**

So **two memory accesses are needed**, which makes the system slower.

To make this faster, the system uses the TLB.

TLB



Translation Look aside Buffer (TLB) is nothing but a **special cache used to keep track of recently used transactions.**

Given a virtual address, the processor (CPU) examines the TLB if a page table entry is present (TLB hit) else TLB Miss.

The page number **p** is first searched in the **TLB (Translation Lookaside Buffer)**. The TLB is a **small fast memory that stores recently used page table entries**

TLB hit- This is **very fast** because the page table is not accessed.

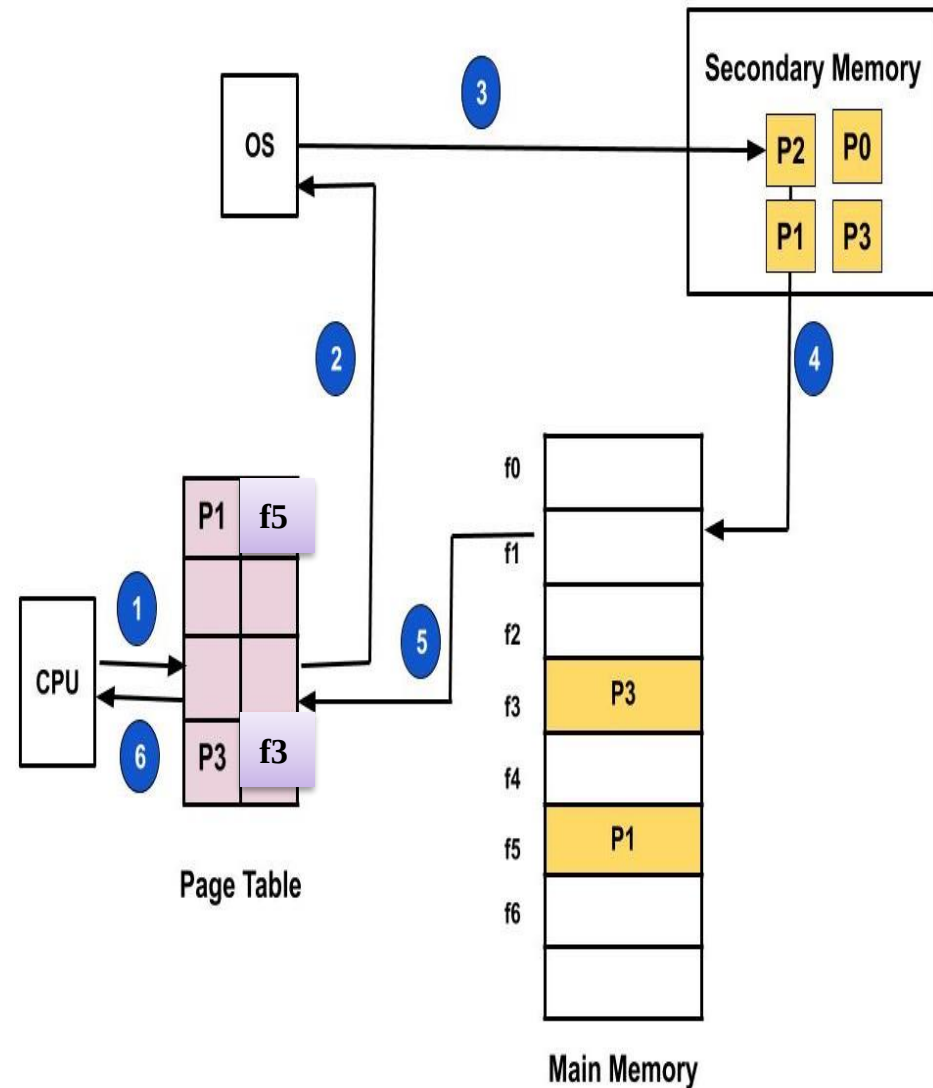
Problem in TLB

- Consider a paging scheme with TLB. If the hit ratio is 75%, then it takes 10 nanoseconds to search in TLB and 200 nanoseconds to access the memory. If the page is not available in TLB then you must access page table and then access the page in memory. Calculate the Effective Access Time (EAT).
- **The Effective Access Time (EAT) is given by the formula:**
- **$EAT = (1 - p) * (t + m) + p * (t + 2m)$**
- where p = hit ratio t = TLB access time m = Memory access time
- Given, $p = 0.75$ (hit ratio) $t = 10$ ns (TLB access time) $m = 200$ ns (Memory access time)
- Substituting the values in the formula, we get:
- $EAT = (1 - 0.75) * (10 + 200) + 0.75 * (10 + 2 * 200) = 0.25 * 210 + 0.75 * 410 = 52.5 + 307.5 = 360$ nanoseconds
- **Therefore, the Effective Access Time (EAT) is 360 nanoseconds.**

Demand Paging

- Demand paging is a technique used in virtual memory systems where the pages are brought in the main memory only when required or demanded by the CPU.
- In order to execute any process, it not necessary that the whole process should present in the main memory at the given time. The process can also be executed if only some pages are present in the main memory at any given time.
- To resolve this problem Demand paging concept came into play. This concept says **we should not load any page into the main memory until required or we should keep all the pages in secondary memory until demanded.**
- Suppose we have to execute a process P having four pages as P0, P1, P2, and P3. Currently, in the page table, we have page P1 and P3

- Now, if the CPU wants to access page P2 of a process P, first it will search the page in the page table.
- As the page table does not contain this page so it will be a trap or page fault. As soon as the trap is generated and context switching happens and the control goes to the operating system.
- The OS system will put the process in a waiting/ blocked state. The OS system will now search that page in the backing store or secondary memory.
- The OS will then read the page from the backing store and A free frame is located in MM the requested process is loaded.
- Next, the OS system will update the page table entry accordingly.
- Finally, the control is taken back from the OS and the execution of the process is resumed.



Thrashing

- **Thrashing** is a situation in which a system spends more time in **processing page faults than executing instructions** of processes.
- Swapping between main memory and secondary memory.
- Thrashing **degrades the performance of a system**.
- In order to make space for swapping in the required page, one of the frequently accessed pages is swapped out from main memory.
- **Soon, the page that is swapped out is required for execution and this again results page fault.**
- Thus, a series of page fault occur and swapping becomes a large overhead.
- Thrashing generally occurs in multitasking operating system.
- As in [multitasking](#), there are several processes running at the same time, so memory becomes over-committed -> Leads to thrashing

Thrashing Example

- **MM has 25 frames.**
- If **P1 needs 50 frames** for execution. But **CPU allocates 3** frames in MM, and remaining frame is kept free for upcoming processes.
- In this above scenario too many page fault occurs as CPU has allocated only 3 frames in MM.
- But in OS point of view CPU utilization is low as only 1 Process is present in MM so it adds too more processes P2 and P3 to CPU which further causes too much in efficiency. This is also known as **thrashing**.
- Further, adding more number of processes to the CPU in order to utilize the CPU efficiently is called as Degree of Multi Programming.